Universitatea Ovidius din Constanța
Facultatea de Inginerie Mecanică, Industrială și Maritimă

Sistem de reglare a nivelului într-un rezervor

Scenă 3D (HTML/three.js) · aplicație Python (customtkinter, Playwright, snap7) · PLC Siemens (TIA Portal V16, PLCSIM V16)

Documentație de dezvoltare, testare și validare

Versiunea 1.0 · noiembrie 2026
Autor: _____

Cuprins

1. Introducere 4
1.1 Scopul documentului 4
1.2 Descrierea sistemului 4
1.3 Fișierele proiectului 5
1.4 Mediul de dezvoltare 5
2. Cerințe 6
2.1 Cerințe funcționale 6
2.2 Cerințe nefuncționale și de siguranță 6
3. Modelul procesului și bucla de reglare 7
3.1 Ecuația rezervorului 7
3.2 Regim staționar și liniarizare 7
3.3 Bucla de reglare 7
4. Scena 3D — rezervor-pid.html 8
4.1 Structura fișierului 8
4.2 Modelul și mașina de stări 8
4.3 Interfața externă 8
4.4 Modificările pentru controlul extern 9
5. Aplicația Python — control_nivel_rezervor.py 10
5.1 Instalare 10
5.2 Arhitectura aplicației 10
5.3 Regulatorul PID 10
5.4 Indicatorii de calitate 11
5.5 Clientul paginii (PageClient) 11
5.6 Gateway-ul PLC (S7Link) 11
5.7 Interfața utilizator 12
5.8 Bucla principală 12
5.9 Utilizare pas cu pas 12
6. Programul PLC — TIA Portal V16 și PLCSIM 14
6.1 Proiectul și CPU-ul 14
6.2 Permiterea accesului PUT/GET 14
6.3 Blocul de date Tank_IO 14
6.4 OB30 — regulatorul PID_Compact 15
6.5 OB100 — pornirea 15
6.6 Compilarea, simularea și puntea Ethernet 16
6.7 Monitorizare și acordare automată 16
7. Testare 17
7.1 Strategia de testare 17
7.2 Teste automate 17
7.3 Teste de integrare (manuale) 17
7.4 Teste de sistem 18
7.5 Rezultate obținute la dezvoltare 18

8. Validare 20
8.1 Criterii de acceptanță 20
8.2 Procedura de validare 20
8.3 Fișa de validare 20
8.4 Comparația celor trei regulatoare 21
9. Depanare 22
10. Dezvoltări ulterioare 23
Anexa A. Exemple de mesaje 24

> Dacă cuprinsul apare gol, în Word: click dreapta pe el → **Update Field** → **Update entire table**.

1. Introducere

1.1 Scopul documentului

Documentul descrie dezvoltarea, testarea și validarea unui sistem didactic de reglare automată a nivelului de lichid într-un rezervor. Procesul fizic este înlocuit de o scenă 3D interactivă care rulează în browser. Reglarea poate fi făcută de un regulator PID scris în Python, de regulatorul intern al scenei sau de un automat programabil Siemens simulat în PLCSIM, programat în TIA Portal V16.

Sistemul este gândit pentru laborator: studenții pot studia modelul procesului, influența parametrilor PID, rejecția perturbațiilor și comunicația industrială S7 (PUT/GET), fără echipament fizic. În același timp, arhitectura reproduce o configurație reală de **punere în funcțiune virtuală** (virtual commissioning): un program PLC este verificat pe un model de proces înainte de a fi încărcat pe instalația reală.

1.2 Descrierea sistemului

Sistemul are trei componente, legate printr-o aplicație Python care joacă rol de gateway (Figura 1):

- **Scena 3D** (`rezervor-pid.html`) simulează procesul (rezervor, valvă de umplere, valvă de golire) și oferă panoul operatorului: START, STOP, oprire de urgență (AU), potențiometrul de consemn și valva de golire folosită ca perturbație.
- **Aplicația Python** (`control_nivel_rezervor.py`) deschide scena, citește nivelul și starea, calculează sau preia comanda valvei de umplere și o trimite înapoi. Afișează rezervorul, graficul în timp, indicatorii de calitate și poate înregistra datele în CSV.
- **PLC-ul simulat** (S7-1200/S7-1500 în PLCSIM V16) rulează regulatorul PID_Compact. Schimbă date cu aplicația Python printr-un bloc de date (DB) neoptimizat, accesat prin snap7 cu servicii PUT/GET.

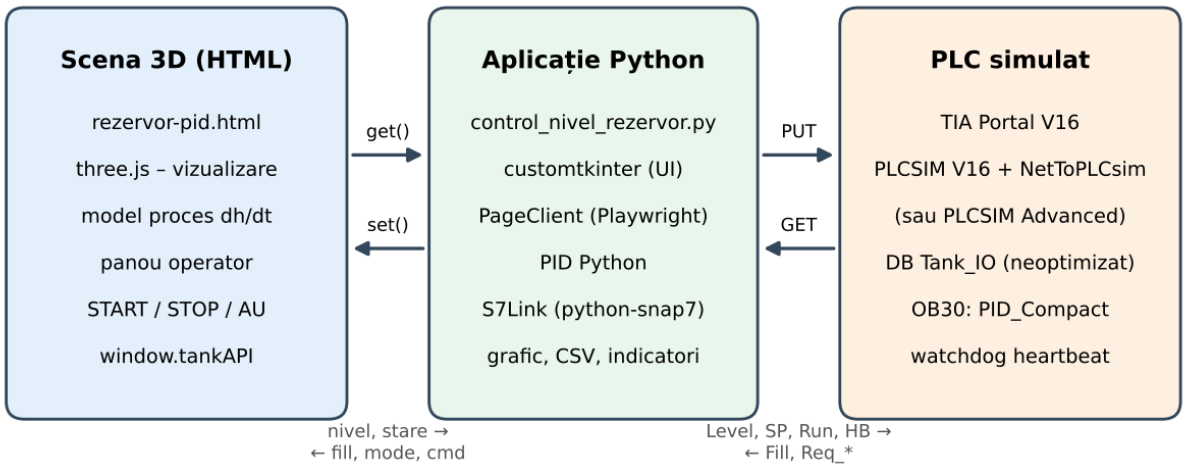


Figura 1. *Arhitectura sistemului și fluxurile de date*

Aplicația Python are trei moduri de reglare, selectabile din interfață. Tabelul 1 rezumă cine calculează comanda în fiecare mod.

Tabelul 1. *Modurile de reglare*

Mod	Unde rulează PID-ul	Rolul aplicației Python	Rolul scenei
PID Python	în aplicația Python (clasa PID)	calculează u și îl trimite ({fill: u})	proces + panou operator (mode = ext)

Mod	Unde rulează PID-ul	Rolul aplicației Python	Rolul scenei
PID scenă	în pagina HTML (funcția step)	trimite consemn, Kp/Ki/Kd, comenzi; monitorizează	proces + regulator (mode = int)
PID PLC	în PLC: OB30 + PID_Compact	gateway scenă ↔ PLC prin snap7	proces + panou operator (mode = ext)

1.3 Fișierele proiectului

Tabelul 2. *Fișierele livrate*

Fișier	Conținut
rezervor-pid.html	Scena 3D: model proces, vizualizare three.js, panou de control, API extern (tankAPI, postMessage, WebSocket), mod de control extern.
control_nivel_rezervor.py	Aplicația Python cu interfață customtkinter: simulare locală, client al paginii (Playwright), PID, gateway PLC (snap7), grafic, indicatori, export CSV.
Tank_IO.scl	Sursa externă TIA Portal pentru DB-ul de schimb (neoptimizat).
test_control.py	Teste automate: model, PID, anti-windup, indicatori, gateway snap7 cu PLC emulat.

1.4 Mediul de dezvoltare

Tabelul 3. *Componente software necesare*

Componentă	Versiune / observații
Windows 10/11 (64 biți)	necesar pentru TIA Portal și PLCSIM
TIA Portal	V16 (STEP 7 Basic/Professional)
S7-PLCSIM	V16 (+ NetToPLCsim pentru acces Ethernet) sau S7-PLCSIM Advanced V3.0+
NetToPLCsim	versiune recentă, gratuit (doar pentru PLCSIM clasic)
Python	3.10 sau mai nou, cu tkinter (inclus în instalarea standard de pe python.org)
customtkinter	pip install customtkinter — interfața grafică
playwright	pip install playwright; apoi: playwright install chromium — controlul paginii
python-snap7	pip install python-snap7 — comunicația S7 (testat cu 3.x)
Browser / internet	scena încarcă three.js r128 de pe cdnjs.cloudflare.com

2. Cerințe

2.1 Cerințe funcționale

Tabelul 4. *Cerințe funcționale*

ID	Cerință
RF-01	Nivelul trebuie reglat automat la un consemn între 0 și 100 % stabilit de operator.
RF-02	Regulatorul trebuie să fie de tip PID, cu Kp, Ki, Kd modificabile în timpul funcționării.
RF-03	Operatorul trebuie să poată porni (START), opri (STOP) și opri de urgență (AU) instalația; ieșirea din AU se face doar prin RESET.
RF-04	La STOP valva de umplere se închide; la AU valva de umplere se închide și valva de golire se deschide complet.
RF-05	Valva de golire trebuie să poată fi modificată ca perturbație, iar regulatorul să o compenseze.
RF-06	Aplicația Python trebuie să poată regla nivelul cu PID propriu, cu PID-ul scenei sau cu PID-ul din PLC.
RF-07	Aplicația Python trebuie să afișeze nivelul, consemnul și comanda în timp real și să calculeze indicatorii de calitate.
RF-08	Datele trebuie să poată fi înregistrate și exportate în CSV.
RF-09	PLC-ul trebuie să poată cere START/STOP/AU (de exemplu protecție la nivel prea mare).
RF-10	Aplicația trebuie să funcționeze și fără scenă și fără PLC (simulare locală).

2.2 Cerințe nefuncționale și de siguranță

Tabelul 5. *Cerințe nefuncționale*

ID	Cerință
RN-01	Perioada de schimb de date scenă ↔ Python ↔ PLC: cel mult 100 ms.
RN-02	La pierderea comunicației cu regulatorul extern, valva de umplere se închide în cel mult 1 s (watchdog în scenă).
RN-03	La pierderea heartbeat-ului de la gateway, PLC-ul semnalează CommFault și trece regulatorul în inactiv în cel mult 2 s.
RN-04	Comanda valvei este limitată la [0, 100] %; regulatorul are anti-windup.
RN-05	Gateway-ul scrie în PLC doar zona proprie din DB (octeții 0…14) și nu suprascrie variabilele scrise de PLC.
RN-06	Interfața rămâne responsivă: comunicația rulează în fire de execuție separate.

3. Modelul procesului și bucla de reglare

3.1 Ecuația rezervorului

Rezervorul este modelat prin bilanțul de volum. Debitul de intrare este proporțional cu deschiderea valvei de umplere u ∈ [0, 1], iar debitul de ieșire respectă legea lui Torricelli, fiind proporțional cu deschiderea valvei de golire d ∈ [0, 1] și cu rădăcina pătrată a nivelului h [%]:

$$dh/dt = 8 \cdot u - 1{,}2 \cdot d \cdot \sqrt{h}\,, \qquad h \in [0,\ 100]\ \%$$

Constanta 8 %/s este viteza maximă de creștere a nivelului (valva de umplere complet deschisă), iar 1,2 este coeficientul de golire. Modelul este identic în scenă (funcția `step()`), în aplicația Python (clasa `TankModel`) și în testele automate, astfel încât rezultatele obținute în simularea locală sunt comparabile cu cele din scenă.

3.2 Regim staționar și liniarizare

În regim staționar dh/dt = 0, deci comanda necesară pentru a menține nivelul h_0 este $u_0 = 1{,}2 \cdot d \cdot \sqrt{h_0} / 8$. Pentru h_0 = 60 % și d = 0,2 rezultă $u_0 \approx 0{,}232$ (23,2 %); pentru d = 0,5, $u_0 \approx 0{,}581$. Aceste valori se regăsesc pe graficul din Figura 3 și sunt un prim test de validare a modelului.

Prin liniarizare în jurul punctului (h_0, u_0) se obține un sistem de ordinul I cu constanta de timp $T = 2\sqrt{h_0} / (1{,}2 \cdot d)$ și factorul de amplificare K = 8·T. Pentru h_0 = 60 % și d = 0,2: T ≈ 64,5 s. Constanta de timp mare față de viteza de umplere arată că procesul se comportă aproape ca un integrator, de unde importanța componentei integrale și a limitării anti-windup.

3.3 Bucla de reglare

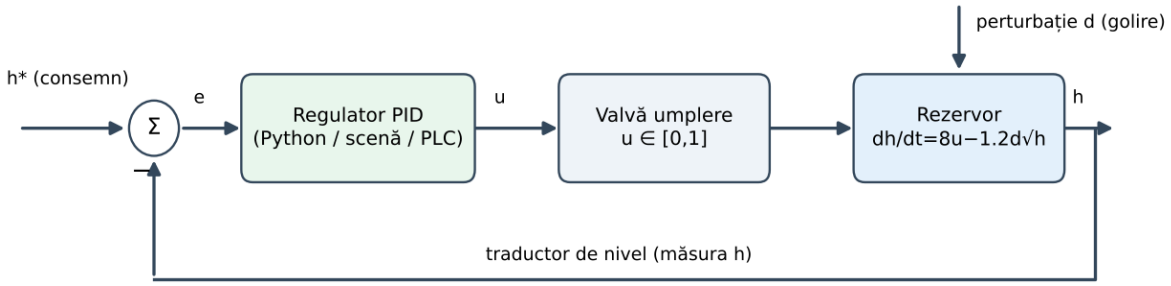


Figura 2. *Schema bloc a sistemului de reglare automată a nivelului*

Eroarea este normalizată la scara 0…1: e = (h* − h)/100. Ieșirea regulatorului este u = Kp·e + Ki·∫e dt + Kd·de/dt, limitată la [0, 1]. Toate cele trei implementări (Python, scenă, PLC) folosesc aceeași scară, deci aceleași valori ale câștigurilor dau comportări comparabile. Excepție face PID_Compact din TIA Portal, care lucrează cu eroarea în procente și cu forma Gain/Ti/Td. Corespondența aproximativă este: Gain ≈ Kp/100, Ti ≈ Kp/Ki, Td ≈ Kd/Kp.

4. Scena 3D — rezervor-pid.html

4.1 Structura fișierului

Scena este un singur fișier HTML, fără server: stil, marcaj și script sunt incluse. Singura dependență externă este biblioteca three.js r128 (de pe cdnjs). Tabelul 6 prezintă secțiunile scriptului.

Tabelul 6. *Secțiunile scriptului din scenă*

Secțiune	Conținut
Stare și model	obiectul S (consemn, câștiguri, nivel, comandă, stare), funcțiile step(dt) și cmd(c)
Scena 3D	renderer WebGL cu umbre, cameră orbitală (mouse/rotiță), mediu de reflexie generat procedural, podea de beton
Echipamente	rezervor din sticlă cu capace și inele, picioare, traductor de nivel, scară gradată, indicatori consemn/nivel, conducte, flanșe, valve cu volan, tavă de colectare, lampă de semnalizare
Lichid	coloană de apă scalată după nivel, suprafață cu unde, jet de umplere și jet de golire animate
UI	panoul de control: butoane, cursoare, grafic consemn/nivel, conexiune WebSocket
API extern	window.tankAPI.get()/set(), postMessage, client WebSocket opțional
Buclă	requestAnimationFrame: 5 pași de integrare pe cadru, actualizare scenă, UI și grafic

4.2 Modelul și mașina de stări

Funcția `step(dt)` integrează modelul prin metoda Euler. Bucla de animație o apelează de 5 ori pe cadru, cu dt limitat la 0,1 s. Dacă tab-ul nu este vizibil, browserul încetinește animația, deci și timpul simulat.

Funcția step() — model și regulator intern / extern

```
function step(dt){
  let dEff=S.drain;
  if(S.state==='RUN'&&S.ext){
    // regulator extern (ex. Python): u vine prin {fill}; watchdog 1 s -> u=0
    if(performance.now()-S.extT>1000)S.uExt=0;
    S.u=S.uExt;S.i=0;S.pe=(S.sp-S.h)/100;
  }else if(S.state==='RUN'){
    const e=(S.sp-S.h)/100;
    S.i=S.ki>0?clamp(S.i+e*dt,-1/S.ki,1/S.ki):0;
    const d=(e-S.pe)/dt; S.pe=e;
    S.u=clamp(S.kp*e+S.ki*S.i+S.kd*d,0,1);
  }else{S.u=0;if(S.state==='ESTOP')dEff=1;}
  S.h=clamp(S.h+(8*S.u-1.2*dEff*Math.sqrt(S.h))*dt,0,100);
  S.dEff=dEff;
}
```

Mașina de stări are trei stări: **STOP** (valvă umplere închisă), **RUN** (reglare activă) și **ESTOP** (umplere închisă, golire deschisă complet). Tranzițiile permise sunt STOP → RUN (start), RUN → STOP (stop), oricare → ESTOP (estop) și ESTOP → STOP (reset).

Funcția cmd() — tranzițiile de stare

```
function cmd(c){
  if(c==='start'&&S.state==='STOP'){S.state='RUN';S.i=0;S.pe=(S.sp-S.h)/100;}
  else if(c==='stop'&&S.state==='RUN')S.state='STOP';
  else if(c==='estop')S.state='ESTOP';
  else if(c==='reset'&&S.state==='ESTOP')S.state='STOP';
}
```

4.3 Interfața externă

Scena poate fi comandată din exterior pe trei căi, toate folosind aceleași mesaje JSON: apel direct `window.tankAPI.set(m)` / `window.tankAPI.get()` (folosit de aplicația Python prin Playwright), `postMessage` (când pagina este inclusă într-un iframe) și un client WebSocket opțional (câmpul de adresă din panou).

Tabelul 7. *Mesaje acceptate de tankAPI.set()*

Cheie	Tip	Efect
sp / setpoint	număr 0…100	consemnul de nivel [%]
kp, ki, kd	număr	câștigurile PID-ului intern
drain	număr 0…1	deschiderea valvei de golire (perturbație)
cmd	"start" | "stop" | "estop" | "reset"	comandă pentru mașina de stări
mode	"ext" | "int"	ext: comanda u vine din exterior; int: PID-ul intern
fill	număr 0…1	comanda externă a valvei de umplere (doar în mode = ext)

Tabelul 8. *Telemetria returnată de tankAPI.get()*

Cheie	Semnificație
t	marca de timp [ms] (Date.now)
level	nivelul curent [%]
sp	consemnul [%]
kp, ki, kd	câștigurile curente
drain	valva de golire [0…1]
fill	comanda aplicată valvei de umplere [0…1]
state	RUN | STOP | ESTOP
mode	ext | int

4.4 Modificările pentru controlul extern

Pentru modurile „PID Python” și „PID PLC”, scena a fost extinsă cu un mod de control extern. În `mode = ext`, starea RUN aplică direct comanda primită (`fill`), iar PID-ul intern este ținut resetat. Un **watchdog de 1 s** închide valva dacă nu mai sosesc comenzi. La deconectarea WebSocket-ului scena revine automat la PID-ul intern, iar textul de stare afișează „· PID extern” cât timp modul este activ. Perioada telemetriei WebSocket a fost redusă de la 200 ms la 100 ms.

Funcția apply() — preluarea mesajelor externe

```
function apply(m){
  for(const k of['sp','kp','ki','kd','drain'])if(typeof
m[k]==='number')S[k]=k==='drain'?clamp(m[k],0,1):m[k];
  if(typeof m.setpoint==='number')S.sp=clamp(m.setpoint,0,100);
  if(m.mode==='ext'){S.ext=true;S.extT=performance.now();}else if(m.mode==='int')S.ext=false;
  if(typeof m.fill==='number'){S.uExt=clamp(m.fill,0,1);S.extT=performance.now();}
  if(m.cmd)cmd(m.cmd);
}
```

5. Aplicația Python — control_nivel_rezervor.py

5.1 Instalare

Instalarea dependențelor și pornirea

```
pip install customtkinter playwright python-snap7
playwright install chromium
python control_nivel_rezervor.py
```

> Fișierele `control_nivel_rezervor.py` și `rezervor-pid.html` trebuie ținute în același folder: aplicația propune implicit pagina de lângă script.

5.2 Arhitectura aplicației

Tabelul 9. *Clasele aplicației*

Clasă	Responsabilitate
TankModel	modelul rezervorului (identic cu scena), folosit în simularea locală și în teste
PID	regulator PID discret: derivată pe măsură filtrată, anti-windup prin integrare condiționată, saturație [0, 1]
StepMetrics	indicatori de calitate la ultimul salt de consemn: timp de creștere, suprareglaj, timp de stabilizare, eroare staționară
PageClient	client al paginii: deschide scena în Chromium (Playwright) și apelează tankAPI.get()/set() la 50 ms, într-un fir separat
S7Link	gateway către PLC: scrie/citește DB-ul de schimb prin snap7 la 50 ms, într-un fir separat; heartbeat
ParamRow	element de interfață: etichetă + câmp numeric + cursor sincronizate
App	fereastra principală (customtkinter): panou, desen rezervor, grafic, bucla _tick() la 50 ms

Interfața tkinter nu este thread-safe, de aceea comunicația cu scena și cu PLC-ul rulează în fire de execuție separate. Firele trimit evenimente (telemetrie, stare conexiune, date PLC) într-o coadă `queue.Queue`, golită de bucla `_tick()` la fiecare 50 ms. Comenzile spre scenă trec printr-o coadă de ieșire a `PageClient`, iar datele spre PLC printr-un dicționar protejat cu `threading.Lock` din `S7Link`.

5.3 Regulatorul PID

Implementarea are trei îmbunătățiri față de PID-ul simplu din scenă. Derivata se calculează pe măsură, nu pe eroare, deci un salt de consemn nu produce un vârf de comandă. Derivata este filtrată cu un filtru de ordinul I (constanta tf = 0,3 s). Integrarea se oprește când ieșirea este saturată și eroarea ar împinge-o și mai mult în saturație (anti-windup).

Metoda PID.update()

```
    def update(self, sp, y, dt):
        if dt <= 0:
            dt = 1e-3
        e = (sp - y) / 100.0
        # derivata pe măsură, filtrată
        if self.prev_y is None:
            d_raw = 0.0
        else:
            d_raw = -(y - self.prev_y) / 100.0 / dt
        a = dt / (self.tf + dt) if self.tf > 0 else 1.0
        self.d += a * (d_raw - self.d)
        self.prev_y = y

        i_new = self.i + e * dt
        u_try = self.kp * e + self.ki * i_new + self.kd * self.d
        saturated_hi = u_try > self.u_max and e > 0
        saturated_lo = u_try < self.u_min and e < 0
        if not (saturated_hi or saturated_lo):
            self.i = i_new
```

```
        self.p_term = self.kp * e
        self.i_term = self.ki * self.i
        self.d_term = self.kd * self.d
        return clamp(self.p_term + self.i_term + self.d_term, self.u_min, self.u_max)
```

5.4 Indicatorii de calitate

Clasa `StepMetrics` pornește o nouă măsurătoare la fiecare schimbare a consemnului mai mare de 0,25 %, dacă instalația este în RUN. Indicatorii sunt: **timpul de creștere** (10–90 % din amplitudinea saltului), **suprareglajul** (depășirea maximă raportată la amplitudine), **timpul de stabilizare** (ultima ieșire din banda ±2 % din amplitudine, cu minimum 0,2 %; raportat după 3 s petrecute continuu în bandă) și **eroarea staționară** (media erorii pe ultimele 5 s).

5.5 Clientul paginii (PageClient)

Un browser nu poate accepta conexiuni de rețea, așa că aplicația Python nu se poate conecta direct la o pagină deschisă de utilizator. `PageClient` deschide el însuși pagina (fișier local sau URL, de exemplu GitHub Pages) într-un Chromium controlat prin Playwright. Așteaptă apariția `window.tankAPI`, apoi, la fiecare 50 ms, trimite comenzile din coadă și citește telemetria. Fereastra rămâne interactivă pentru operator. La deconectare, clientul trimite `{mode: "int"}` și închide browserul.

Bucla clientului paginii (extras)

```
                page.wait_for_function("() => window.tankAPI !== undefined", timeout=20000)
                self.connected = True
                self.events.put(("connected", True))
                self.events.put(("status", f"conectat la {url}"))
                while not self._stop.is_set():
                    while True:                       # comenzi în așteptare -> pagină
                        try:
                            msg = self.out.get_nowait()
                        except queue.Empty:
                            break
                        page.evaluate("m => window.tankAPI.set(m)", msg)
                    tel = page.evaluate("() => window.tankAPI.get()")
                    if isinstance(tel, dict):
                        self.events.put(("tel", tel))
                    page.wait_for_timeout(self.POLL_MS)
```

5.6 Gateway-ul PLC (S7Link)

La fiecare 50 ms, `S7Link` scrie un bloc de 15 octeți (DB, octeții 0…14) cu nivelul, consemnul, perturbația, heartbeat-ul și biții Run/EStop. Apoi citește 6 octeți (15…20) cu cererile PLC-ului, ieșirea PID și CommFault. Zona scrisă și zona citită nu se suprapun, deci gateway-ul nu suprascrie niciodată valorile scrise de PLC (cerința RN-05). Valorile Real sunt codate big-endian (formatul S7), cu funcțiile `set_real` / `get_real` din `snap7.util`. După 5 erori consecutive de comunicație legătura se închide și se raportează eroarea.

Ciclul de schimb al gateway-ului (extras)

```
                set_real(wbuf, 0, float(o["level"]))
                set_real(wbuf, 4, float(o["sp"]))
                set_real(wbuf, 8, float(o["drain"]) * 100.0)
                set_int(wbuf, 12, hb)
                set_bool(wbuf, 14, 0, bool(o["run"]))
                set_bool(wbuf, 14, 1, bool(o["estop"]))
                try:
                    cli.db_write(db, self.W_START, wbuf)
                    r = cli.db_read(db, self.R_START, self.R_SIZE)
                    errors = 0
                except Exception as e:  # noqa
                    errors += 1
                    if errors >= 5:
                        raise
```

```
                    continue
                self.events.put(("plc", {
                    "req_start": get_bool(r, 0, 0), "req_stop": get_bool(r, 0, 1), "req_estop": get_bool(r,
0, 2),
                    "fill": get_real(r, 1), "comm_fault": get_bool(r, 5, 0), "hb": hb}))
```

Aplicația detectează fronturile crescătoare ale biților Req_Start, Req_Stop și Req_EStop și le transformă în comenzi pentru scenă. Astfel, programul PLC poate porni sau opri instalația, de exemplu pentru protecția la nivel prea mare.

5.7 Interfața utilizator

Fereastra are în stânga panoul de control (derulabil) și în dreapta desenul rezervorului și graficul. Elementele panoului, de sus în jos:

- **Proces**: Simulare locală / Scenă 3D; câmpul cu pagina scenei (fișier sau URL), butonul „…” pentru alegerea fișierului și „Conectare la scenă”.
- **Regulator**: PID Python / PID scenă / PID PLC; secțiunea PLC cu IP, Port, Rack, Slot, DB nr. și „Conectare PLC”.
- **Comenzi**: START, STOP, AU (devine RESET AU în oprire de urgență) și eticheta de stare colorată.
- **Parametri**: nivel țintă, Kp, Ki, Kd, valva de golire; fiecare are cursor și câmp numeric (se confirmă cu Enter).
- **Valori instantanee**: nivel, eroare, comandă u, termenii P / I / D.
- **Răspuns la ultimul salt de consemn**: indicatorii din 5.4.
- **Înregistrare**: pornire/oprire înregistrare la 10 Hz și export CSV (separator „;”).

Desenul rezervorului arată nivelul, linia de consemn, deschiderea valvelor (culoarea trece din roșu în verde) și lampa de stare. Graficul afișează ultimele 60 s: consemnul (galben), nivelul (albastru) și comanda u (verde).

5.8 Bucla principală

Metoda `_tick()` rulează la 50 ms. Golește coada de evenimente, determină nivelul (din model sau din telemetrie), calculează sau preia comanda u în funcție de mod, actualizează datele trimise spre PLC, indicatorii, istoricul și desenul. Parametrii modificați în scenă sunt preluați automat în aplicație (sincronizare bidirecțională). Pentru a evita „lupta” dintre cele două interfețe, o valoare primită din scenă este ignorată timp de 0,8 s după ce aplicația a trimis-o ea însăși.

Tabelul 10. *Calculul comenzii u în funcție de sursă și mod*

Sursă \ Mod	PID Python	PID scenă	PID PLC
Simulare locală	PID.update() pe modelul local	(nu se aplică — se folosește PID Python)	Fill din PLC / 100 aplicat modelului local
Scenă 3D	PID.update() pe nivelul din scenă → {fill}	fill raportat de scenă (doar afișare)	Fill din PLC / 100 → {fill}

5.9 Utilizare pas cu pas

1. Porniți aplicația. Fără alte setări, sursa este „Simulare locală”, iar regulatorul „PID Python”.
2. Apăsați START și urmăriți urcarea nivelului spre consemn. Modificați consemnul, câștigurile și valva de golire și observați efectul pe grafic și în indicatori.
3. Apăsați „Conectare la scenă”. Se deschide fereastra Chromium cu scena 3D, iar sursa trece pe „Scenă 3D”.
4. Alegeți modul de reglare. În „PID Python” scena afișează „· PID extern”.

5. Pentru PLC: completați IP, port, rack, slot, DB, apăsați „Conectare PLC” și alegeți „PID PLC” (vezi capitolul 6).
6. Pentru o măsurătoare: porniți înregistrarea, faceți saltul de consemn, așteptați stabilizarea, opriți înregistrarea și exportați CSV-ul.

6. Programul PLC — TIA Portal V16 și PLCSIM

6.1 Proiectul și CPU-ul

1. Creați un proiect nou în TIA Portal V16 (Create new project) și adăugați un CPU, de exemplu S7-1200 CPU 1214C DC/DC/DC sau S7-1500 CPU 1511-1 PN, cu firmware suportat de PLCSIM V16.
2. În proprietățile interfeței PROFINET setați o adresă IP, de exemplu 192.168.0.1 / 255.255.255.0. PLCSIM clasic nu folosește această adresă pentru comunicația externă, dar ea este necesară pentru PLCSIM Advanced.

6.2 Permiterea accesului PUT/GET

snap7 folosește serviciile PUT/GET ale protocolului S7. La CPU-urile S7-1200/1500 aceste servicii sunt dezactivate implicit.

1. CPU → Properties → General → **Protection & Security** → nivel de acces „**Full access (no protection)**”.
2. Tot acolo, la **Connection mechanisms**, bifați „**Permit access with PUT/GET communication from remote partner**”.

> Fără bifa PUT/GET, conectarea snap7 reușește, dar citirile și scrierile DB-ului eșuează (eroare de acces / „function refused by CPU”).

6.3 Blocul de date Tank_IO

DB-ul de schimb trebuie să fie **neoptimizat**: snap7 accesează datele prin adrese absolute (număr DB + offset), care există doar la blocurile cu „Optimized block access” dezactivat. Cea mai sigură cale este importul sursei livrate:

1. În arborele proiectului: Program blocks → **External source files** → Add new external file → selectați `Tank_IO.scl`.
2. Click dreapta pe sursă → **Generate blocks from source**. Se creează DB-ul „Tank_IO” cu atributul S7_Optimized_Access = FALSE.
3. Verificați în proprietățile DB-ului numărul (de exemplu DB1) și faptul că „Optimized block access” nu este bifat. Compilați și deschideți DB-ul: coloana Offset trebuie să corespundă cu Tabelul 11.

Tabelul 11. *Structura DB-ului Tank_IO*

Offset	Nume	Tip	Sens	Descriere
0.0	Level	Real	GW → PLC	nivel măsurat [%]
4.0	Setpoint	Real	GW → PLC	consemn [%]
8.0	Drain	Real	GW → PLC	valva de golire / perturbație [%]
12.0	Heartbeat	Int	GW → PLC	contor incrementat de gateway la fiecare ciclu
14.0	Run	Bool	GW → PLC	instalația este în RUN
14.1	EStop	Bool	GW → PLC	oprire de urgență activă
14.2–14.7	Spare_14_x	Bool	—	rezervă (aliniază Req_* la octetul 15)
15.0	Req_Start	Bool	PLC → GW	front crescător → START
15.1	Req_Stop	Bool	PLC → GW	front crescător → STOP
15.2	Req_EStop	Bool	PLC → GW	front crescător → oprire de urgență
16.0	Fill	Real	PLC → GW	ieșirea PID = comanda valvei de umplere [%]
20.0	CommFault	Bool	PLC	heartbeat pierdut > 2 s

Offset	Nume	Tip	Sens	Descriere
22.0	HbCnt	Int	PLC intern	cicluri fără schimbare de heartbeat
24.0	HbLast	Int	PLC intern	ultima valoare a heartbeat-ului

6.4 OB30 — regulatorul PID_Compact

1. Add new block → Organization block → **Cyclic interrupt** (OB30), limbaj SCL, perioada **100 ms**.
2. Din Instructions → Technology → PID Control → Compact PID, trageți **PID_Compact** în OB30 și acceptați crearea DB-ului de instanță „PID_Compact_1”.
3. Scrieți codul de mai jos (îl puteți copia ca atare).

OB30 (SCL)

```
// OB30 - Reglare nivel rezervor (100 ms)
// 1) Watchdog comunicatie: heartbeat-ul de la gateway trebuie sa se schimbe (20 x 100 ms = 2 s)
IF "Tank_IO".Heartbeat <> "Tank_IO".HbLast THEN
    "Tank_IO".HbLast := "Tank_IO".Heartbeat;
    "Tank_IO".HbCnt := 0;
ELSIF "Tank_IO".HbCnt < 1000 THEN
    "Tank_IO".HbCnt := "Tank_IO".HbCnt + 1;
END_IF;
"Tank_IO".CommFault := "Tank_IO".HbCnt > 20;

// 2) Regulator PID: inactiv (iesire 0) cand instalatia e oprita, la AU sau fara comunicatie
"PID_Compact_1"(Setpoint := "Tank_IO".Setpoint,
                Input    := "Tank_IO".Level,
                Reset    := NOT "Tank_IO".Run OR "Tank_IO".EStop OR "Tank_IO".CommFault,
                Output   => "Tank_IO".Fill);

// 3) Protectie: nivel prea mare -> cerere de oprire de urgenta
"Tank_IO".Req_EStop := "Tank_IO".Level > 95.0;
```

Configurarea PID_Compact (dublu click pe instrucțiune → Configuration):

Tabelul 12. *Configurarea PID_Compact*

Parametru	Valoare	Observații
Controller type	General, unitate %	mărime reglată în procente
Invert control logic	nebifat	u crescut → nivel crescut
Mode after CPU restart	Activate Mode, Automatic	vezi și OB100
Input / Output	Input (Real) / Output (Real)	nu se folosesc Input_PER / Output_PER / Output_PWM
Process value limits	0,0 … 100,0 %	limitele nivelului
Output value limits	0,0 … 100,0 %	domeniul valvei de umplere
PID parameters	inițial: Gain = 0,05, Ti = 17 s, Td = 0,2 s	echivalentul aproximativ al Kp = 5, Ki = 0,3, Kd = 1; apoi acordare automată (6.7)

> Echivalența este aproximativă: PID_Compact folosește eroarea în procente (deci Gain ≈ Kp/100) și are ponderări proprii pentru P și D (b, c) și filtru pe derivată. Valorile inițiale sunt doar un punct de plecare.

6.5 OB100 — pornirea

Când Reset trece din TRUE în FALSE, PID_Compact intră în modul de funcționare memorat în parametrul Mode. Pentru a intra în modul automat, setați Mode la pornirea CPU-ului: Add new block → Organization block → **Startup** (OB100), limbaj SCL:

OB100 (SCL)

```
// OB100 - Startup
"PID_Compact_1".Mode := 3;   // 3 = automatic
```

6.6 Compilarea, simularea și puntea Ethernet

1. Compilați: click dreapta pe CPU → Compile → Hardware and software (rebuild all). Corectați eventualele erori.
2. Porniți simularea: butonul **Start simulation** (PLCSIM V16). În Extended download alegeți interfața PLCSIM, încărcați programul și puneți CPU-ul în **RUN**.
3. PLCSIM V16 clasic nu are interfață Ethernet, deci snap7 nu se poate conecta direct. Porniți **NetToPLCsim**. La pornire, dacă portul 102 este ocupat, acceptați oprirea serviciului care îl ține.
4. În NetToPLCsim: Add → alegeți la „Network IP Address” adresa PC-ului (127.0.0.1 dacă aplicația Python rulează pe același calculator), la „Plcsim IP Address” instanța PLCSIM care rulează, TSAP/rack-slot implicit → OK → **Start Server**. Starea trebuie să indice conexiunea cu PLCSIM.
5. În Windows Firewall permiteți TCP 102 dacă aplicația Python rulează pe alt calculator.
6. În aplicația Python: IP = adresa din NetToPLCsim, Port = 102, Rack = 0, Slot = 1, DB nr. = numărul DB-ului Tank_IO → **Conectare PLC**. Eticheta trebuie să afișeze „conectat … DB1” și apoi „ieșire PID … ✓ comunicație OK”.

> **Alternativă — S7-PLCSIM Advanced (V3.0+)**: creați o instanță cu „PLCSIM Virtual Eth. Adapter”, cu IP-ul din configurația hardware, încărcați programul în ea și conectați aplicația Python direct la acel IP, port 102. NetToPLCsim nu mai este necesar. Setările PUT/GET și DB-ul neoptimizat rămân obligatorii.

6.7 Monitorizare și acordare automată

Creați o tabelă de monitorizare (Watch and force tables → Add new watch table) cu variabilele Tank_IO.Level, Setpoint, Fill, Run, EStop, Heartbeat, CommFault și HbCnt. Cu „Monitor all” trebuie să vedeți Heartbeat crescând continuu și Level urmărind scena.

Deoarece procesul este simulat în scenă, se poate folosi și **acordarea automată** a PID_Compact (Technology objects → PID_Compact_1 → Commissioning):

1. Puneți instalația în RUN (START în scenă), cu nivelul stabil și departe de consemn.
2. În fereastra Commissioning: Measurement → Start; Tuning mode = **Pretuning** → Start. PID_Compact aplică un salt de comandă, identifică procesul și calculează parametrii.
3. După pretuning puteți rula **Fine tuning**, care excită o oscilație controlată în jurul consemnului și rafinează parametrii.
4. Upload PID parameters → încărcați parametrii în proiect, comparați apoi indicatorii cu cei obținuți cu PID Python (capitolul 8).

7. Testare

7.1 Strategia de testare

Testarea se face pe trei niveluri: **teste unitare** pe componentele de calcul (model, PID, indicatori), **teste de integrare** pe interfețe (Python ↔ scenă, Python ↔ PLC) și **teste de sistem**, care verifică funcționarea completă din punctul de vedere al operatorului. Testele unitare și testul de integrare cu PLC emulat sunt automate (`test_control.py`). Restul se execută manual, după procedurile de mai jos.

7.2 Teste automate

Rulare (în folderul aplicației)

```
python test_control.py
```

Testul T-I1 pornește un server S7 local (snap7) pe portul 1102, cu un DB de aceeași structură ca Tank_IO, și un „PLC emulat” care rulează un regulator PI și watchdog-ul la 100 ms. Gateway-ul S7Link este verificat exact ca în funcționarea reală.

Tabelul 13. *Teste automate și rezultatele obținute în mediul de dezvoltare*

ID	Test	Criteriu	Rezultat
T-U1	Echilibru model	la $u_0 = 1{,}2 \cdot d \cdot \sqrt{h_0}/8$ nivelul rămâne 60 % (3 zecimale)	TRECUT
T-U2	Limite model	nivelul rămâne în [0, 100] %	TRECUT
T-U3	Răspuns la treaptă 10 → 60 %	suprareglaj < 10 %, |e| final < 0,5 %	TRECUT
T-U4	Rejecția perturbației	golire 0,2 → 0,5: nivelul revine la consemn (|e| < 0,5 %)	TRECUT
T-U5	Saturație / anti-windup	$u \in [0, 1]$; integrala rămâne limitată după 100 s de saturație	TRECUT
T-U6	Fără lovitură derivativă	termenul D = 0 la salt de consemn cu măsură constantă	TRECUT
T-I1	Gateway + PLC emulat	nivel final 60 ± 3 % în 30 s; CommFault la deconectare	TRECUT

Rezultatele „TRECUT” au fost obținute la dezvoltare (Python 3.13, python-snap7 3.4, Linux), cu modulele grafice înlocuite de stub-uri. Pe calculatorul de laborator testele se rulează din nou, cu toate dependențele instalate, iar rezultatul se trece în fișa de validare.

7.3 Teste de integrare (manuale)

Tabelul 14. *Teste de integrare*

ID	Scop	Pași	Rezultat așteptat
T-I2	Conectare la scenă	Conectare la scenă cu fișierul local	se deschide Chromium cu scena; eticheta: „conectat la file:///…”
T-I3	Telemetrie	mutați cursorul „Nivel țintă” în scenă	valoarea se actualizează în aplicație în < 1 s
T-I4	Comenzi spre scenă	START / STOP / AU / RESET AU din aplicație	starea scenei urmează comenzile; lampa își schimbă culoarea
T-I5	Mod extern	regulator „PID Python”, START	scena afișează „ÎN FUNCȚIUNE · PID extern”; valva urmează comanda Python
T-I6	Watchdog scenă	în „PID Python”, RUN: închideți forțat aplicația Python	în ≤ 1 s valva de umplere se închide
T-I7	Conectare PLC	PLCSIM în RUN + NetToPLCsim; Conectare PLC	„conectat … DB1”; Heartbeat crește în watch table
T-I8	Date spre PLC	modificați consemnul și golirea	Tank_IO.Setpoint și Drain se actualizează în watch table

ID	Scop	Pași	Rezultat așteptat
T-I9	Date din PLC	forțați Tank_IO.Fill în watch table (Reset activ)	valoarea apare în eticheta PLC a aplicației
T-I10	Watchdog PLC	în RUN, apăsați „Deconectare PLC”	după ~2 s CommFault = TRUE, Fill = 0
T-I11	Cerere din PLC	consemn 98 %, RUN în „PID PLC”	la nivel > 95 % scena trece în OPRIRE DE URGENȚĂ

7.4 Teste de sistem

Tabelul 15. *Teste de sistem*

ID	Scenariu	Criteriu de trecere
T-S1	Simulare locală, PID Python: START, salt 10 → 60 %	indicatorii respectă criteriile din Tabelul 17
T-S2	Scenă, PID Python: salt 60 → 30 % și înapoi la 60 %	idem; graficul din scenă și cel din aplicație coincid
T-S3	Scenă, PID scenă: aceleași câștiguri	răspuns asemănător cu T-S2 (diferențe mici: PID-ul scenei derivă eroarea, deci are lovitură derivativă)
T-S4	Scenă, PID PLC: salt 10 → 60 %, apoi perturbație 0,2 → 0,5	nivelul ajunge și revine la consemn; eroare staționară < 1 %
T-S5	Oprire de urgență în fiecare mod	valva de umplere se închide imediat, golirea se deschide complet, ieșirea din AU numai prin RESET
T-S6	Înregistrare și export CSV	fișierul se deschide în Excel (separator „;”), conține toate coloanele
T-S7	Funcționare continuă 30 min, PID PLC	fără deconectări, fără erori, interfață responsivă

7.5 Rezultate obținute la dezvoltare

Figura 3 prezintă răspunsul simulat (model local, pas 10 ms) pentru trei seturi de parametri: un salt de consemn de la 10 % la 60 %, urmat la t = 70 s de o perturbație (valva de golire de la 0,2 la 0,5). La începutul saltului comanda este saturată la 100 %, iar nivelul urcă cu viteza maximă de 8 %/s. Diferențele dintre acordări apar mai ales în apropierea consemnului și la rejecția perturbației.

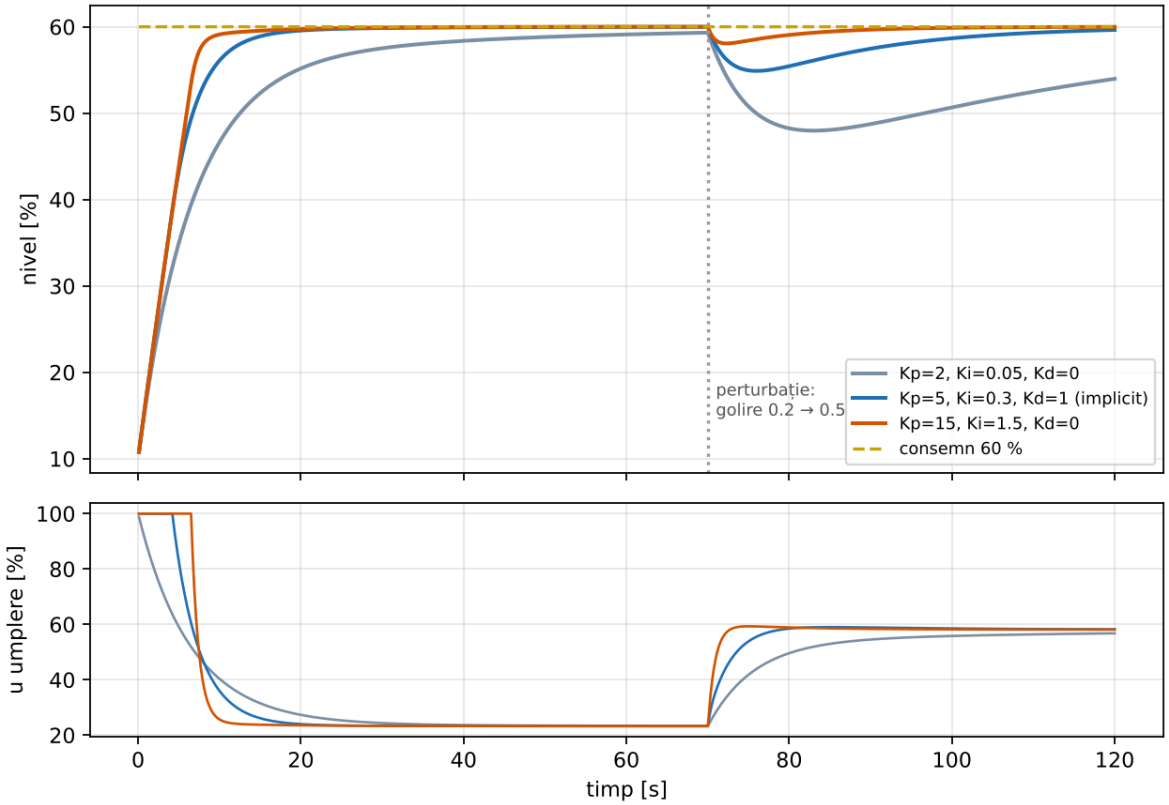


Figura 3. *Răspunsul la salt de consemn și la perturbație pentru trei acordări (simulare)*

Tabelul 16. *Indicatorii de calitate la saltul 10 → 60 % (simulare, din StepMetrics)*

Acordare	Timp de creștere	Suprareglaj	Timp de stabilizare (±2 %)	Eroare staționară
Kp = 2, Ki = 0,05, Kd = 0	18,9 s	0,0 %	55,9 s	+0,73 %
Kp = 5, Ki = 0,3, Kd = 1 (implicit)	8,4 s	0,0 %	15,4 s	+0,01 %
Kp = 15, Ki = 1,5, Kd = 0	6,1 s	0,0 %	9,5 s	+0,00 %

Acordarea lentă nu atinge consemnul în fereastra de 70 s și rejectă slab perturbația (nivelul coboară până la ~48 %). Acordările implicită și agresivă stabilizează fără suprareglaj, deoarece saturația comenzii limitează viteza, iar anti-windup-ul împiedică acumularea integralei. Acordarea agresivă rejectă cel mai bine perturbația, dar într-un proces real, cu zgomot de măsură și întârzieri, ar produce o comandă agitată. Ca bună practică, se alege un compromis apropiat de acordarea implicită.

În plus, la dezvoltare au fost verificate cu pagina reală, încărcată în Chromium prin Playwright: conectarea, trecerea în modul extern, comanda START și reglarea nivelului de către PID-ul Python (testele T-I2, T-I4, T-I5). Testele cu PLCSIM real (T-I7 … T-I11, T-S4, T-S7) se execută pe calculatorul cu TIA Portal.

8. Validare

8.1 Criterii de acceptanță

Validarea confirmă că sistemul răspunde scopului didactic și cerințelor din capitolul 2. Criteriile de performanță se verifică la saltul de consemn 10 → 60 % cu golirea 0,2, iar cele de siguranță prin testele de întrerupere.

Tabelul 17. *Criterii de acceptanță*

Criteriu	Valoare acceptată	Cerință	Test
Suprareglaj	≤ 10 %	RF-01, RF-02	T-S1, T-S2, T-S4
Timp de stabilizare (±2 %)	≤ 30 s	RF-01	T-S1, T-S2, T-S4
Eroare staționară	≤ 1 %	RF-01	T-S1 … T-S4
Rejecția perturbației 0,2 → 0,5	revenire la ±2 % în ≤ 40 s	RF-05	T-U4, T-S4
Perioada schimbului de date	≤ 100 ms	RN-01	T-I3, T-I7
Watchdog scenă	valva închisă în ≤ 1 s	RN-02	T-I6
Watchdog PLC	CommFault în ≤ 2,5 s	RN-03	T-I10, T-I1
Oprire de urgență	imediat, în toate modurile	RF-03, RF-04	T-S5
Teste automate	7/7 trecute	—	test_control.py

8.2 Procedura de validare

1. Instalați dependențele (5.1) și rulați `python test_control.py`; toate testele trebuie să treacă.
2. Executați testele de integrare T-I2 … T-I6 (scena) și T-I7 … T-I11 (PLC).
3. Pentru fiecare mod (Python, scenă, PLC): consemn 10 %, așteptați stabilizarea, porniți înregistrarea, faceți saltul la 60 %, așteptați 60 s, apoi schimbați golirea la 0,5 și mai așteptați 60 s. Opriți înregistrarea și exportați CSV-ul.
4. Notați indicatorii afișați de aplicație și verificați-i în CSV (de exemplu cu un grafic în Excel).
5. Executați testele de siguranță T-S5 și T-I6, T-I10 în fiecare mod relevant.
6. Completați fișa de validare. Sistemul este validat dacă toate criteriile din Tabelul 17 sunt îndeplinite.

8.3 Fișa de validare

Tabelul 18. *Fișa de validare (de completat)*

Criteriu	PID Python	PID scenă	PID PLC	Conform (DA/NU)
Parametri utilizați				—
Timp de creștere [s]				—
Suprareglaj [%]				
Timp de stabilizare [s]				
Eroare staționară [%]				
Rejecția perturbației [s]				
Watchdog (T-I6 / T-I10)		—		
Oprire de urgență (T-S5)				
Teste automate (7/7)		—	—	

Data: _____ Validat de: _____ Semnătura: _____

8.4 Comparația celor trei regulatoare

Cu aceleași câștiguri (Kp = 5, Ki = 0,3, Kd = 1), PID-ul Python și cel al scenei au răspunsuri apropiate. Diferențele provin din derivata pe eroare a PID-ului scenei (vârf de comandă la salt de consemn) și din saturația diferită a integralei. PID_Compact are altă structură (ponderări, filtre, limitări interne), deci comparația corectă se face după acordarea automată: pretuning, eventual fine tuning, apoi aceleași teste. Diferențele observate și explicația lor pot face obiectul unei teme de laborator.

9. Depanare

Tabelul 19. *Probleme frecvente*

Simptom	Cauză probabilă	Rezolvare
„pagina nu expune window.tankAPI”	three.js nu s-a încărcat (fără internet) sau fișier greșit	verificați conexiunea la internet și calea fișierului; deschideți pagina manual în browser
„lipsește Playwright”	pachetul sau browserul nu este instalat	pip install playwright; playwright install chromium
Scena nu afișează „· PID extern”	se folosește pagina veche, fără suport mode/fill	folosiți rezervor-pid.html actualizat
„eroare PLC: … connection refused / timeout”	NetToPLCsim oprit, IP/port greșit, firewall	porniți serverul NetToPLCsim, verificați IP-ul, portul 102 și firewall-ul
Conectat, dar erori la citire/scriere	PUT/GET nebifat sau DB optimizat	vezi 6.2 și 6.3; recompilați și încărcați
Valori aberante în DB	număr DB greșit sau structură diferită	verificați numărul DB-ului și offset-urile (Tabelul 11)
CommFault = TRUE permanent	gateway-ul nu scrie (deconectat) sau OB30 nu rulează	verificați „Conectare PLC” și existența OB30 cu 100 ms
PID_Compact nu reglează (Fill = 0)	Reset activ sau Mode ≠ 3	Run = TRUE, EStop = FALSE, CommFault = FALSE; verificați OB100 / starea în Commissioning
Nivelul crește prea încet în scenă	fereastra scenei este minimizată (animația încetinește)	țineți fereastra scenei vizibilă
Portul 102 ocupat la NetToPLCsim	serviciu Siemens sau sesiune online TIA	acceptați oprirea serviciului sau folosiți alt port (ex. 1102) și treceți-l în aplicație

10. Dezvoltări ulterioare

- Panou HMI WinCC (Basic/Advanced) în TIA Portal, conectat la același PLCSIM, pentru comparație cu panoul scenei.
- Transmiterea câștigurilor din aplicație direct în PID_Compact (Retain.CtrlParams), cu conversia Kp/Ki/Kd ↔ Gain/Ti/Td.
- Mod cu broker MQTT, astfel încât scena să poată fi deschisă de studenți în propriul browser (de exemplu de pe GitHub Pages).
- Zgomot de măsură și întârziere de transport configurabile în model, pentru studiul filtrării și al robustеții.
- Comunicație OPC UA (S7-1500) ca alternativă la PUT/GET.

Anexa A. Exemple de mesaje

Mesaje JSON

```
// Python -> scena (tankAPI.set)
{"mode": "ext"}
{"sp": 60, "kp": 5, "ki": 0.3, "kd": 1, "drain": 0.2}
{"cmd": "start"}
{"fill": 0.2317}

// scena -> Python (tankAPI.get)
{"t": 1794236073645, "level": 59.962, "sp": 60, "kp": 5, "ki": 0.3, "kd": 1,
 "drain": 0.2, "fill": 0.232, "state": "RUN", "mode": "ext"}
```

Exemplu de fișier CSV exportat

```
t_s;consemn_pct;nivel_pct;u_umplere;d_golire;kp;ki;kd;stare;sursa;regulator
12.30;60.00;58.214;0.3312;0.200;5.0;0.3;1.0;RUN;scene;plc
```